SPRING DOCS

Spring Security

Authentication, Authorization, Filter Chains & JWT

Java Agentic AI — Knowledge Base Reference

1. Core Concepts

Term	Meaning
Authentication	Verifying WHO the user is (login credentials, tokens)
Authorization	Verifying WHAT the authenticated user is allowed to do (roles/permissions)
Principal	The currently authenticated user (or system)
GrantedAuthority	A single permission/role held by the principal, e.g. ROLE_ADMIN
SecurityContext	Holds the Authentication object for the current thread/request

2. The Security Filter Chain

Spring Security works by inserting a chain of servlet Filters in front of the DispatcherServlet. Each filter has one responsibility (CSRF checks, authentication, authorization, exception translation, etc.) and requests pass through them in a defined order before reaching the application.

- SecurityContextPersistenceFilter — loads/saves the SecurityContext (session-based apps).
- UsernamePasswordAuthenticationFilter — handles form-login credential submission.
- BasicAuthenticationFilter — handles HTTP Basic auth headers.
- A custom JWT filter (not built-in) is typically added here for token-based auth.
- FilterSecurityInterceptor / AuthorizationFilter — enforces access-control decisions last.

3. Basic Configuration (Modern, Java Config)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())                       // often disabled for
stateless REST APIs
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
```

```
    }
}
```

Note: *Since Spring Security 5.7+, the fluent SecurityFilterChain @Bean style replaces the older WebSecurityConfigurerAdapter class-extension approach, which is now deprecated.*

4. UserDetailsService

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return org.springframework.security.core.userdetails.User.builder()
            .username(user.getUsername())
            .password(user.getPasswordHash())
            .roles(user.getRole())
            .build();
    }
}
```

5. Password Encoding

Passwords must never be stored in plaintext. BCryptPasswordEncoder is the standard choice — it's a slow, salted, adaptive hashing algorithm specifically designed to resist brute-force and rainbow-table attacks.

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String hashed = encoder.encode("myPassword123");     // includes a random salt internally
boolean matches = encoder.matches("myPassword123", hashed); // used during login
verification
```

6. Method-Level Security

```
@EnableMethodSecurity // enables @PreAuthorize/@PostAuthorize/@Secured
@Configuration
public class MethodSecurityConfig { }

@Service
public class AccountService {

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteAccount(Long id) { ... }
```

```
    @PreAuthorize("#username == authentication.name") // caller can only access their own
data
    public Account getAccount(String username) { ... }

    @PostAuthorize("returnObject.owner == authentication.name")
    public Document getDocument(Long id) { ... }
}
```

7. JWT-Based Stateless Authentication

For REST APIs, session-based auth doesn't scale well across stateless, horizontally-scaled services. JSON Web Tokens (JWT) let the server issue a signed, self-contained token after login; subsequent requests carry it in the Authorization header, and the server verifies its signature without needing shared session storage.

```
// Login endpoint issues the token
String token = Jwts.builder()
    .setSubject(user.getUsername())
    .claim("roles", user.getRoles())
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 3600_000))
    .signWith(SignatureAlgorithm.HS256, secretKey)
    .compact();

// Custom filter validates the token on every request
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain)
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtUtil.validateToken(token)) {
                String username = jwtUtil.getUsername(token);
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(username, null, authorities);
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

Aspect	Session-based Auth	JWT-based Auth
State	Server-side session storage	Stateless — all claims in the token itself
Scalability	Needs sticky sessions or shared session store	Scales horizontally with no shared state
Revocation	Easy — just invalidate the session	Harder — needs a blocklist or short expiry + refresh tokens

Aspect	Session-based Auth	JWT-based Auth
Payload exposure	Opaque session ID only	Claims are base64-encoded, readable (not encrypted) unless using JW

8. CORS and CSRF

- **CORS** (Cross-Origin Resource Sharing) controls which origins a browser is allowed to make requests from — configured via @CrossOrigin or a global CorsConfigurationSource bean.
- **CSRF** (Cross-Site Request Forgery) protection is essential for session-cookie-based, browser-driven state-changing requests; it's commonly disabled for stateless, token-authenticated REST APIs since there's no ambient browser session to forge.

9. Quick Interview Recap

- Authentication vs Authorization? Authentication verifies identity (who you are); authorization verifies permission (what you can do) once identity is established.
- Why disable CSRF for REST APIs? Stateless token-based APIs don't rely on browser-managed session cookies, which is the attack vector CSRF protection exists to guard against.
- Why use BCrypt over MD5/SHA-256 for passwords? BCrypt is deliberately slow and includes a random salt, making brute-force and rainbow-table attacks impractical — MD5/SHA are fast general-purpose hashes unsuitable for password storage.
- How does a JWT filter authenticate a request? It intercepts the Authorization header, verifies the token signature/expiry, extracts the username/roles, and manually populates the SecurityContext so downstream authorization checks see an authenticated principal.